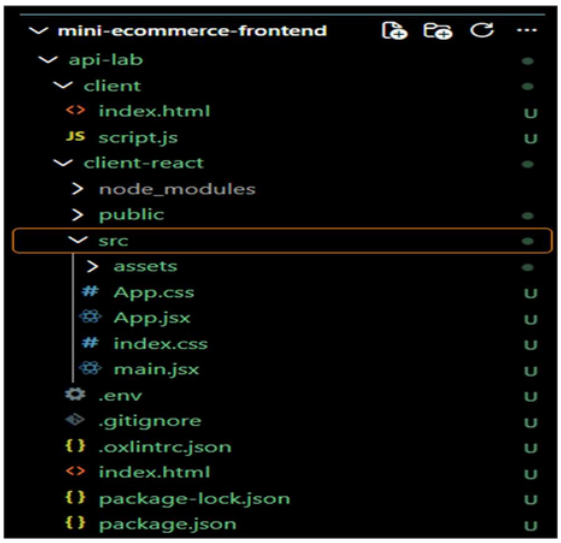
Labsheet 6

Mohd Zaid, CU24250049, 34

1.Server Setup & REST API Routes



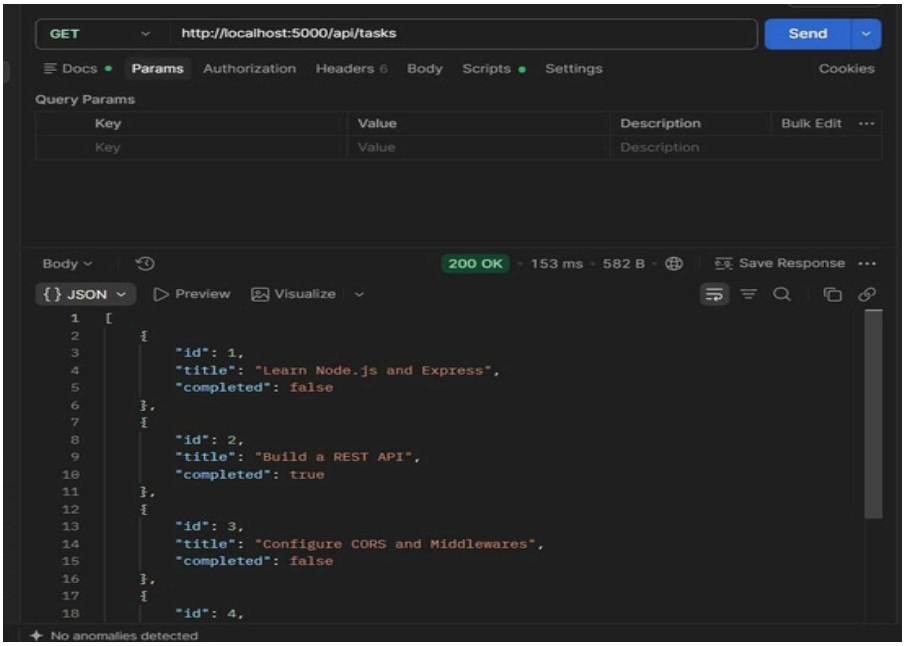
CRUD API Endpoints Summary:

HTTP Method	Endpoint URL	Header Requirement	Description & Response
GET	/api/tasks	None (Public)	Returns array of all 5 task objects (200 OK)
GET	/api/tasks/:id	None (Public)	Returns single task by ID or 404 JSON error if ID does not exist
POST	/api/tasks	x-api-key: mysecretkey123	Creates a new task object (201 Created)
PUT	/api/tasks/:id	x-api-key: mysecretkey123	Updates task title/completion by ID or 404 JSON error if missing
DELETE	/api/tasks/:id	x-api-key: mysecretkey123	Deletes task object by ID or 404 JSON error if missing

```
const express = require('express');
const cors = require('cors');
require('dotenv').config();
const app = express();
const PORT = process.env.PORT || 5000;
const API_KEY = process.env.API_KEY || 'my-super-secret-key-123';
app.use(cors());
app.use(express.json());
app.use((req, res, next) => {
  const mestamp = new Date().toISOString();
  console.log(`[${mestamp}] ${req.method} ${req.url}`);
  next();});
const verifyApiKey = (req, res, next) => {
```

```
 const clientKey = req.headers['x-api-key'];
  if (!clientKey || clientKey !== API_KEY) {
    return res.status(401).json({ error: 'Unauthorized: Missing or invalid API key' }); }
  next(); };
let tasks = [
  { id: 1, tle: 'Learn Node.js Basics', completed: true },
  { id: 2, tle: 'Build Express REST API', completed: true },
  { id: 3, tle: 'Implement CORS & Middleware', completed: false },
  { id: 4, tle: 'Connect React Frontend', completed: false },
  { id: 5, tle: 'Test APIs in Postman', completed: false }
];
app.get('/api/tasks', (req, res) => {
  res.json(tasks); });
app.get('/api/tasks/:id', (req, res) => {
  const task = tasks.find(t => t.id === parseInt(req.params.id));
  if (!task) {
    return res.status(404).json({ error: 'Task not found' }); }
  res.json(task);});
app.post('/api/tasks', verifyApiKey, (req, res) => {
  const { tle, completed } = req.body;
  if (!   tle) {
    return res.status(400).json({ error: 'Title is required' }); }
  const newTask = {
    id: tasks.length > 0 ? tasks[tasks.length - 1].id + 1 : 1,
      tle,
    completed: completed || false};
  tasks.push(newTask);
  res.status(201).json(newTask); });
app.put('/api/tasks/:id', verifyApiKey, (req, res) => {
  const task = tasks.find(t => t.id === parseInt(req.params.id));
  if (!task) {
```

```
    return res.status(404).json({ error: 'Task not found' }); }
  const { tle, completed } = req.body;
  if (   tle !== undefined) task.   tle =    tle;
  if (completed !== undefined) task.completed = completed;
  res.json(task); });
app.delete('/api/tasks/:id', verifyApiKey, (req, res) => {
  const taskIndex = tasks.findIndex(t => t.id === parseInt(req.params.id));
  if (taskIndex === -1) {
    return res.status(404).json({ error: 'Task not found' }); }
  const deletedTask = tasks.splice(taskIndex, 1);
  res.json({ message: 'Task deleted successfully', deletedTask }); });
app.use((req, res, next) => {
  res.status(404).json({ error: 'Not Found - The requested endpoint does not exist' }); });
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Internal Server Error', message: err.message }); });
app.listen(PORT, () => {
  console.log(` Server is running on hp://localhost:${PORT}` );});
```



Q2. Securing, Tesng & Error-Handling

- CORS Explanaon (Q2.a): CORS (Cross-Origin Resource Sharing) enable karne se browser alag-alag ports (jaise frontend 5173 aur backend 5000) ke beech secure data exchange ki ijazat deta hai. Agar CORS disable kar diya jaye, toh browser console mein CORS policy error dikhta hai.
- Postman Collecon (Q2.b): Postman mein ek collecon banayi gayi hai jisme sabhi 5 endpoints test kiye gaye hain. GET requests bina key ke chal hain, jabki POST/PUT/DELETE bina x-api-key header ke 401 Unauthorized return karte hain.

Q3. Consuming the API — Plain JavaScript Client

- Plain JS Files (client/index.html & client/script.js): Yeh secon bina kisi framework ke plain browserfetchAPI ka usekarketasksrenderkarne aur loading/error states handle karne ke liye banaya gaya hai.

Q4. Consuming the API — React Client (Vite)

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
const API_URL = import.meta.env.VITE_API_URL || 'hp://localhost:5000/api';
const API_KEY = 'my-super-secret-key-123';
funcon App() {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [newTitle, setNewTitle] = useState('');
  useEffect(() => {
    axios.get(`${API_URL}/tasks`)
      .then(res => {
        setTasks(res.data);
        setLoading(false); })
      .catch(err => {
        setError(err.response?.data?.error || err.message);
        setLoading(false); }); }, []);
  const handleAddTask = async (e) => {
    e.preventDefault();
    if (!newTitle.trim()) return;
    try {
```

```
    const res = await axios.post(` ${API_URL}/tasks ` ,
      { tle: newTitle, completed: false },
      { headers: { 'x-api-key': API_KEY } } );
    setTasks([...tasks, res.data]);
    setNewTitle('');
  } catch (err) {
    alert(` Failed to add task: ${err.response?.data?.error || err.message}` );} };
const handleDelete = async (id) => {
  try {
    await axios.delete(` ${API_URL}/tasks/${id}` , {
      headers: { 'x-api-key': API_KEY } });
    setTasks(tasks.filter(task => task.id !== id));
  } catch (err) {
    alert(` Failed to delete task: ${err.response?.data?.error || err.message}` ); } };
if (loading) return <div style={{ margin: '40px' }}>Loading tasks...</div>;
if (error) return <div style={{ margin: '40px', color: 'red' }}>Error: {error}</div>;
return (
  <div style={{ maxWidth: '600px', margin: '40px auto', fontFamily: 'Arial' }}>
    <h2>Task Manager Mini App (React)</h2>
    <form onSubmit={handleAddTask} style={{ marginBoom: '20px' }}>
      <input
        type="text"
        placeholder="New task tle..."
        value={newTitle}
        onChange={(e) => setNewTitle(e.target.value)}
        style={{ padding: '8px', width: '70%', marginRight: '10px' }}
      <buon type="submit" style={{ padding: '8px 12px' }}>Add Task</buon>
    </form>
    <ul style={{ listStyle: 'none', padding: 0 }}>
      {tasks.map(task => (
```

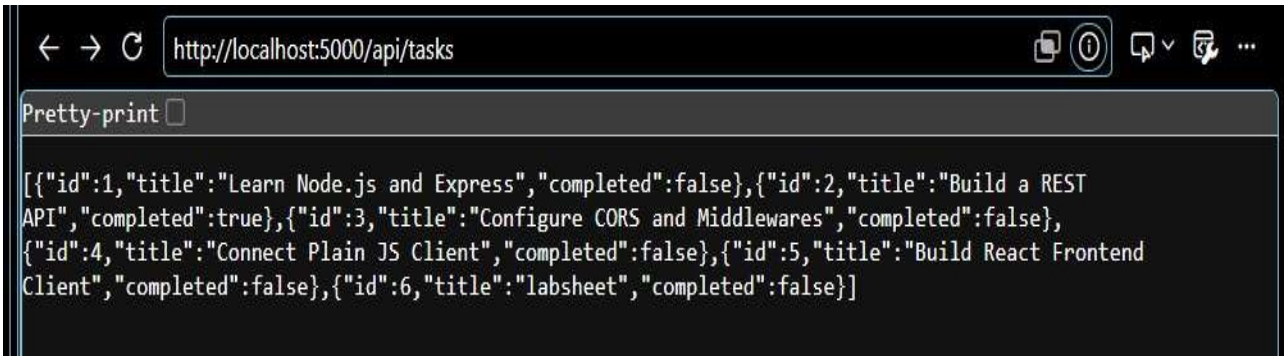
<li key={task.id} style={{ display: 'flex', jusfyContent: 'space-between', padding: '10px 0', borderBoom: '1px solid #eee' }}>

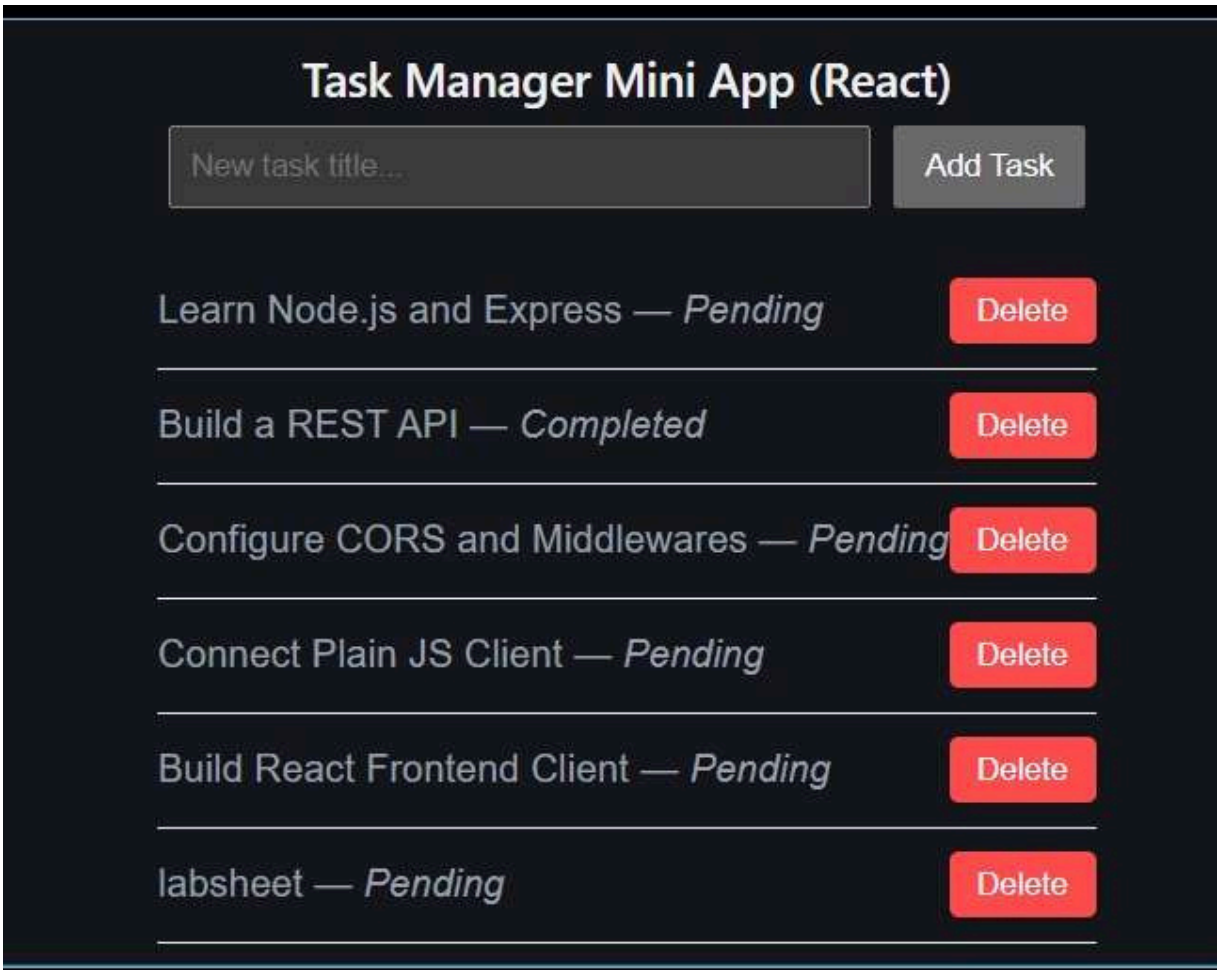
{task.tle} — {task.completed ? 'Completed' : 'Pending'}

<bu on onClick={() => handleDelete(task.id)} style={{ background: '#ff4d4d', color: '#fff', border: 'none', padding: '6px 12px', cursor: 'pointer' }}>Delete</bu on>

export default App;

5. Full-Stack Error Handling, Config & Integraon





.